

RAPPORT TECHNIQUE

Projet Spring Boot Bancaire

Application de gestion bancaire
Spring Boot 3 • React 19 • H2 • Docker

Technologie backend	Spring Boot 3 / Java
Frontend	React 19 / Vite
Base de données	H2 (in-memory)
Conteneurisation	Docker + Docker Compose
Port backend	8080
Port frontend	5173 (dev) / 80 (prod)

Mai 2026

1. Introduction

Ce rapport présente le projet BankApp Spring, une application de gestion bancaire développée dans le cadre du module Spring Boot. Elle permet la gestion des clients, des comptes bancaires et des transactions financières, avec une interface web moderne et une API REST complète.

L'application illustre l'utilisation des technologies modernes Java/Spring coté backend et React coté frontend, le tout conteneurisé via Docker.

2. Architecture Générale

2.1 Vue d'ensemble

L'application adopte une architecture client-serveur claire en deux niveaux :

- Frontend React (port 5173 en dev, 80 en prod) : interface utilisateur SPA
- Backend Spring Boot (port 8080) : API REST, logique métier, persistance
- Base de données H2 in-memory : données volatiles, idéale pour les démos
- Docker Compose : orchestre les deux services en production

2.2 Structure du projet

Dossier	Rôle
backend/src/main/java/com/bankapp/model/	Entités JPA (Account, Client, Transaction, AppUser)
backend/src/main/java/com/bankapp/controller/	Contrôleurs REST (Auth, Account, Client, Transaction, Transfer)
backend/src/main/java/com/bankapp/service/	Logique métier (AuthService, TransferService, etc.)
backend/src/main/java/com/bankapp/repository/	Interfaces Spring Data JPA
backend/src/main/java/com/bankapp/dto/	Objets de transfert de données (AuthResponse, TransferRequest, etc.)
backend/src/main/resources/	application.properties + data.sql (données initiales)
frontend/src/	App.jsx (composant principal), services/api.js, styles.css

3. Couche Domaine (Modèle de Données)

3.1 Entités principales

Le modèle de données est composé de quatre entités JPA principales :

AppUser

Représente un utilisateur de l'application (administrateur ou client). Possède un username, un mot de passe en clair, un rôle (ADMIN ou CLIENT) et une référence optionnelle vers un Client.

Client

Représente un client bancaire avec son username unique et son numéro de téléphone. Chaque client est lié à un ou plusieurs comptes bancaires.

Account

Compte bancaire identifié par un numéro unique (ex : ACC-MEHDI-001), associé à un Client et doté d'un solde BigDecimal.

Transaction

Enregistrement d'un mouvement financier (DEBIT ou CREDIT) sur un compte, avec montant et date de réalisation.

3.2 Relations entre entités

Entité source	Entité cible	Type de relation
AppUser	Client	ManyToOne (optionnel)
Account	Client	ManyToOne (obligatoire)
Transaction	Account	ManyToOne (obligatoire)

4. API REST

4.1 Endpoints disponibles

Méthode	Endpoint	Description
POST	/api/auth/login	Authentification (username + password)
POST	/api/auth/register	Inscription (crée AppUser + Client associé)
POST	/api/auth/logout	Déconnexion (stateless, vide)
GET	/api/accounts	Liste tous les comptes
POST	/api/accounts	Crée un nouveau compte
PUT	/api/accounts/{id}	Modifie un compte
DELETE	/api/accounts/{id}	Supprime un compte

Méthode	Endpoint	Description
GET	/api/clients	Liste tous les clients
POST	/api/clients	Crée un nouveau client
GET	/api/transactions/recent	Retourne les transactions récentes
POST	/api/transfers	Exécute un transfert entre comptes

4.2 Authentification

L'authentification est implémentée de manière stateless. Après un login réussi, le backend retourne un objet `AuthResponse` contenant le username, le rôle et l'identifiant du client associé. Le frontend stocke cet objet dans le `localStorage` du navigateur.

Note importante : les mots de passe sont stockés en clair dans cette version. Il s'agit d'un projet pédagogique ; en production, il faudrait utiliser `BCrypt` ou `Argon2`.

5. Logique Métier

5.1 Transfert de fonds (`TransferService`)

La fonctionnalité de transfert est le cœur métier de l'application. Elle permet d'envoyer de l'argent entre comptes en identifiant le destinataire de trois façons :

- Par numéro de compte (ex : ACC-MEHDI-001)
- Par numéro de téléphone du client (ex : 0600000002)
- Par username du client (ex : mehdi), insensible à la casse

Le service vérifie : que les comptes source et destination sont différents, que le montant est strictement positif, et que le solde source est suffisant. Il enregistre ensuite deux transactions (un DEBIT et un CREDIT) dans le même contexte transactionnel (`@Transactional`).

5.2 Gestion des utilisateurs (`AuthService`)

Lors de l'inscription, le service crée simultanément un `Client` (profil bancaire) et un `AppUser` (compte de connexion), les deux partageant le même username. Cela garantit la cohérence entre le compte applicatif et le profil bancaire.

6. Frontend React

6.1 Technologies utilisées

Librairie	Version	Usage
React	19	Framework UI
Vite	6	Bundler et serveur de développement
Axios	1.x	Requêtes HTTP vers l'API
Lucide React	0.4xx	Icônes vectorielles

6.2 Fonctionnalités de l'interface

L'interface est une Single Page Application (SPA) structurée autour d'un unique composant App.jsx qui gère l'état global. Elle offre :

- Page d'authentification avec basculement login / inscription
- Dashboard avec statistiques en temps réel (nombre de comptes, solde global, transactions)
- Formulaire de transfert (par numéro de compte, téléphone ou username)
- Gestion des clients et comptes réservée au rôle ADMIN
- Affichage des transactions récentes et des cartes de comptes
- Gestion des droits par rôle (CLIENT ne voit que ses propres comptes)

6.3 Gestion des droits

Le filtrage des données s'effectue côté client : un utilisateur CLIENT ne voit que les comptes et transactions liés à son clientId. Les formulaires d'administration (ajout client, ajout compte) ne sont affichés que pour le rôle ADMIN.

7. Configuration et Données Initiales

7.1 Configuration Spring Boot

```
spring.datasource.url=jdbc:h2:mem:bankdb
spring.jpa.hibernate.ddl-auto=create
spring.h2.console.enabled=true
```

La base de données H2 est recrée à chaque démarrage (ddl-auto=create). La console H2 est accessible sur <http://localhost:8080/h2-console> pour l'inspection directe des tables.

7.2 Données initiales (data.sql)

Un fichier data.sql insère automatiquement des données de démo au démarrage : un utilisateur admin (admin/admin123), plusieurs clients de test et leurs comptes bancaires avec des soldes prédéfinis.

8. Conteneurisation Docker

8.1 Architecture Docker

Le projet inclut un Dockerfile par service et un docker-compose.yml à la racine. Le backend est basé sur une image JDK 21 slim, et le frontend utilise une image Nginx pour servir les fichiers de build.

8.2 Commandes de démarrage

```
# Démarrage complet (production)
docker compose up --build
```

```
# Développement backend uniquement
cd backend && ./mvnw spring-boot:run
```

```
# Développement frontend uniquement
cd frontend && npm run dev
```

9. Points Forts et Axes d'Amélioration

9.1 Points forts

- Architecture propre avec séparation des responsabilités (model / repository / service / controller)
- Transactions ACID garanties via @Transactional sur les opérations bancaires sensibles
- Flexibilité du transfert : trois modes d'identification du destinataire
- Interface réactive avec gestion des rôles (ADMIN vs CLIENT)
- CI/CD via GitHub Actions (.github/workflows/ci-cd.yml)
- Conteneurisation complète avec Docker Compose

9.2 Axes d'amélioration

- Sécurité : implémenter Spring Security avec JWT et hachage des mots de passe (BCrypt)
- Validation : ajouter @Valid et les contraintes Bean Validation sur les DTOs
- Persistance : migrer vers PostgreSQL ou MySQL pour les environnements de production
- Tests : ajouter des tests unitaires et d'intégration (JUnit 5, Mockito, Spring Boot Test)

- Gestion des erreurs : centraliser la gestion des exceptions via `@ControllerAdvice`
- Pagination : paginer les listes de transactions et de comptes pour les grands volumes

10. Conclusion

Le projet BankApp Spring constitue une application bancaire fonctionnelle et bien structurée, illustrant les bonnes pratiques du développement Spring Boot moderne. La séparation claire entre frontend et backend, l'utilisation des annotations Spring et la logique transactionnelle démontrent une maîtrise des concepts fondamentaux du framework.

L'application est prête à évoluer vers un système de production en intégrant la sécurité, une base de données persistante et une couverture de tests complète.